7/03 structures to function

```
struct ABC
{ int data1;
  char s1[20];
};

int main()
{ struct ABC v1;
  f1( v1 );
}
void f1(struct ABC v)
{
}
```

v1 → actual argument

v → formal ..

typedef is used to give short and simple name to structure datatype.
typedef with structure can be used in two ways
1. after structure declaration
2. along with structure declaration

syntax of typedef

typedef existingdatatype newtype;

```
struct Student
{
        int roll;
        char name[20];
        float marks;
};

typedef struct Student STUDENT;

STUDENT var;
```

STUDENT → datatype, var → variable name

this is same as struct Student var;

method2 :

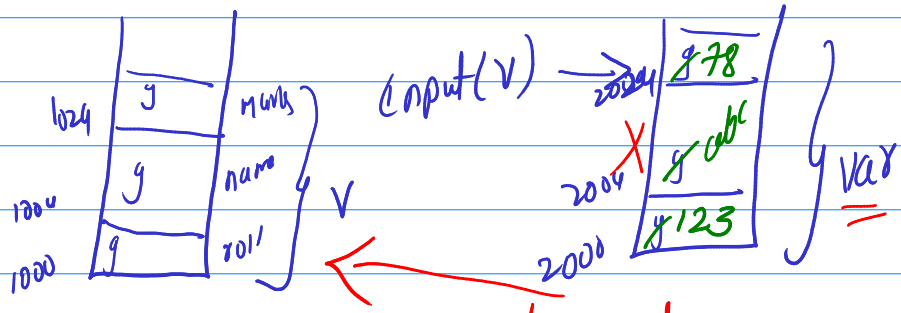
```
typedef struct Student
{       int roll;
        char name[20];
        float marks;
}STUDENT;
```

another name of struct Student

```
struct Student
{
        int roll;
        char name[20];
        float marks;
}Stu;
```

Stu → variable

```
int main( )
{ STUDENT V;
    Input(V);
    Print(V);
}
void Input(STUDENT Var)
{ =
}
void Print(STUDENT Var)
{ =
}
```



this is a call by value
so what ever changes we made at Var is not updated to V, So to overcome either we need to return Var or go with call by reference

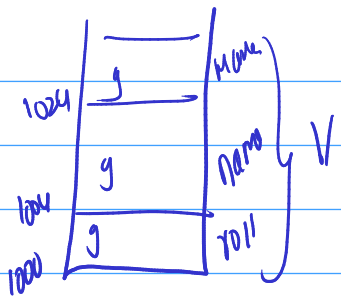
with return type

```
struct Student Input (struct Student Var)
{
    =
    return Var;
}
```

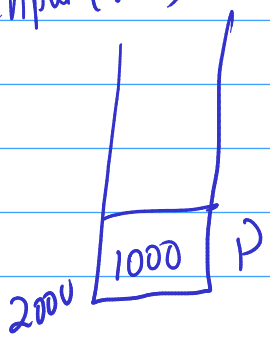
```
Input(&v);


void Input(STUDENT *p)
{


}
```



Input(&V);



refer to array of structure to Fun

	roll	name	marks	
Arr[0]	123	abc	78	Arr
Arr[1]				Arr+1
Arr[2]				Arr+2
Arr[3]				Arr+3
Arr[4]				Arr+4

Input(Arr, 5);

Input(STUDENT *p, int size)

```
{
  int i=0;
  for(i=0; i<5; i++)
  {
    printf("enter roll\n");
    scanf("%d", &p[i].roll);
    scanf("%s", p[i].name);
    scanf("%f", &p[i].marks);
  }
}
```

p

Arr

&p[0].roll

scanf("%d", &p->roll);
scanf("%s", p->name);
scanf("%f", &p->marks);

it always stores at zeroth variable address only.
so to solve it increment p after every iteration or

&((p+i)->roll)